Localizing INT8 Quantization Collapse in EfficientNet-B0 for Edge Poultry Disease Detection

Fatih Jawwad Al Mumtaz
Department of Informatics
University of Darussalam Gontor
Email: fatihalmumtaz76@student.cs.unida.gontor.ac.id

Amirul Al Aziz
Department of Agricultural Industrial Technology
University of Darussalam Gontor
Email: amirulalaziz57@student.tip.unida.gontor.ac.id

Muhammad Iqbal Hidayatullah
Department of Informatics
University of Darussalam Gontor
Email: iqbalhidayatullah4563@gmail.com

***Abstract*—We compare MobileNetV2, ShuffleNetV2, and EfficientNet-B0 for poultry-fecal disease classification on resource-constrained edge devices. The audit of the published 8,770-image benchmark finds 617 near-duplicate pairs, including 94 test images byte-identical to training images; results therefore use an 8,153-image deduplicated split. Dynamic INT8 changes MobileNetV2 and ShuffleNetV2 only slightly but collapses EfficientNet-B0 to 10.42%. Static per-channel QDQ recovers EfficientNet-B0 to 94.56%, while selective ablation indicates strong sensitivity to depthwise-layer granularity. On Raspberry Pi 5, dynamic INT8 is 1.93–2.80× slower than FP32 at four threads, whereas static per-channel QDQ reaches 94.56% at 13.50 ms, 2.34× faster than FP32. These results show that deployment outcomes depend jointly on architecture, runtime, and quantization granularity.**

***Index Terms*—deep learning, poultry disease detection, lightweight CNN, quantization, edge computing, post-training quantization**

I. Introduction

Poultry production is a major agricultural sector (combined broiler, egg, turkey, and chicken value of $67.1 billion in the United States in 2023 [10]). Coccidiosis, Salmonellosis, and Newcastle Disease (NCD) cause major losses [2]. Fecal image analysis enables noninvasive early detection, but laboratory diagnosis remains inaccessible to many small farms [3].

Recent deep learning approaches have achieved high accuracy for poultry fecal disease classification. Dhungana *et al.* reported 99.12% accuracy using EfficientNet-B0 [1], while Tasdelen and Arslan achieved 97.1% with MobileNetV2 [3]. However, these solutions primarily target cloud-based deployment, leaving a critical gap: *how can these models be efficiently deployed on resource-constrained edge devices at farm level?*

Lightweight CNNs—MobileNetV2 [5], ShuffleNetV2 [6], and EfficientNet [7]—are candidates for edge deployment because they need fewer operations. INT8 post-training quantization (PTQ) reduces size by using 8-bit weights [8]. Even on CPUs that expose INT8 acceleration, *dynamic* PTQ can increase latency when the runtime kernel path adds quantize/dequantize work around every convolution; our x86 reference shows a 3.0–16.8× slowdown for dynamic INT8, whereas static QDQ is faster than FP32 on Raspberry Pi 5. Kernel-level attribution is discussed in Sec. IV-C, and [14] explains why PTQ without calibration degrades on depthwise layers.

This paper addresses two research questions: (1) What is the quantization sensitivity of each lightweight architecture under INT8 PTQ on CPU, and where exactly does EfficientNet-B0 fail? (2) What is the accuracy-latency tradeoff for these models under INT8 PTQ on a Raspberry Pi 5? We additionally audit the dataset integrity claim underlying prior work on this benchmark.

The contributions are as follows. First, we audit the FecalFed leak-free claim [9] and identify 617 near-duplicate pairs, including 94 test images byte-identical to training images; all results use the resulting deduplicated split. Second, we characterize EfficientNet-B0 sensitivity to quantization granularity: dynamic INT8 and static per-tensor QDQ collapse to 10.42% and 9.32%, while static per-channel QDQ recovers 94.56%. Third, we characterize the three models on Raspberry Pi 5 across 12 model–quantization–thread configurations, showing that deployment latency depends on the runtime and quantization scheme.

II. Related Work

A. Poultry Disease Detection

Machuve *et al.* introduced the foundational dataset of 6,812 PCR-verified fecal images from Tanzanian poultry farms [2]; subsequent work combined YOLO-based detection with CNN classification [1], [4] or benchmarked architectures for accuracy and training speed [3]. Chi introduced FecalFed, a federated-learning benchmark on this dataset family claiming 8,770 rigorously deduplicated unique images [9]; our audit in Section III-A tests that claim directly. Prior work has not compared lightweight models under quantization with accuracy-latency tradeoff analysis.

B. Lightweight CNN Architectures

MobileNetV2 [5] utilizes inverted residuals with linear bottlenecks and depthwise separable convolutions for efficient

feature extraction. ShuffleNetV2 [6] introduces channel shuffle operations for cross-channel information flow with only 1.26M parameters. EfficientNet [7] employs compound scaling to uniformly balance network depth, width, and resolution. Hoang *et al.* recently compared seven lightweight architectures for plant disease detection, identifying hybrid designs as optimal for edge deployment [11].

C. Model Quantization

INT8 quantization represents weights with 8-bit integers, theoretically achieving 4× compression [8]. ONNX Runtime notes that INT8 without matching kernel support may degrade performance [12], and [14] surveys why PTQ without calibration degrades on depthwise and SE layers.

III. Methodology

A. Dataset

We use the Machuve *et al.* dataset [13] with PCR-verified fecal images from Tanzanian farms, as distributed via HuggingFace `Dianyo/poultry-fecal-fl`. Chi [9] describes this benchmark as 8,770 rigorously deduplicated leak-free images with train/test splits of 7,016/1,754.

We audit that claim with pHash (Hamming ≤ 5) over all 8,770 files and find 617 near-duplicate pairs (all size 2, class-consistent; 393 byte-exact), 303 of which span splits: 147 test–train, 54 test–validation, and 102 train–validation. Of these, 94 test images are byte-identical to a train image and 31 to a validation image.

Splitting precedes deduplication. The publisher train split (7,016) was carved into a stratified 80/20 train/validation partition (seed 42), with the publisher test split (1,754) intact; we then dropped one copy per near-duplicate pair (keeping test $>$ val $>$ train; within a split, lexicographically-first kept). This removed 513 train, 78 validation, and 26 test images, exactly 617, yielding 8,153 at 5,100/1,325/1,728 with zero cross-split survivors (artifacts `phash_dedup.json`, `sterile_split_manifest.json`). Validation is 20.6% of train plus validation rather than exactly 20% because unequal numbers were removed from each side.

The partition contains no cross-split near-duplicate pair detected by this audit, which is not a claim of independence between images, since residual dependence may remain within a split. This removes leakage but does not rebuild a fresh split from the Zenodo originals (6,812 images); the test set remains 98% inherited from the publisher test split. Classes are Coccidiosis, Healthy, Salmonella, and Newcastle Disease; NCD is underrepresented (417/5,100 train, 141/1,728 test). Images are resized to 224×224 and normalized with ImageNet mean/std.

B. Model Architectures

We evaluate the three ImageNet [15]-pretrained architectures of Sec. II-B with the classification head replaced for four classes (3.4M→2.2M for MobileNetV2 after the head swap). Our 4-class models hold 2.2M (MobileNetV2), 1.3M (ShuffleNetV2), and 4.0M (EfficientNet-B0) parameters.

C. Training Protocol

All layers are trainable (torchvision V1 weights, no freezing). Each model is trained once (seed 42) for 50 epochs with Adam (10^{-4}, weight decay 10^{-4}) and cosine annealing, keeping the best-validation checkpoint; batch size 32, input 224×224. Augmentation (train only) is horizontal flip, $\pm 15^{\circ}$ rotation, and 0.2 color jitter; eval preprocessing is resize→tensor→ImageNet normalize. Training used an RTX 4060 (8 GB) with PyTorch 2.5.1/CUDA 12.1.

D. Quantization and Benchmarking (x86)

Models are exported to ONNX (opset 13) and quantized with ONNX Runtime 1.29.0 dynamic per-tensor INT8 (signed QInt8 weights, dynamically quantized activations); per-channel scales use the output-channel axis. The static-QDQ run used the first 100 training images sequentially; because the dataset loader is class-ordered, this calibration set contained only Coccidiosis images and was not class-balanced, so the static-QDQ results are diagnostic evidence from a single-class calibration slice, not a claim of calibration robustness. Static QDQ uses signed QInt8 weights and activations, MinMax calibration, and per-tensor or per-channel weight scales, with no custom graph fusion. The native-Linux rerun, performed with the same ORT model-only protocol, gives dynamic-INT8 slowdowns of 14.73–14.97× for MobileNetV2, 3.22–3.26× for ShuffleNetV2, and 14.08× for EfficientNet-B0 at one and four threads, respectively. These host-reference values are reported separately from the WSL and Raspberry Pi 5 results.

E. Edge Deployment on Raspberry Pi 5

We characterize the same ONNX FP32 and INT8 models on a Raspberry Pi 5 (Broadcom BCM2712, 4× Cortex-A76 at up to 2.4 GHz, 8 GB LPDDR4X, Debian 13 trixie, kernel 6.18.34+rpt-rpi-2712, aarch64, `ondemand` governor, throttled=0x0). ONNX Runtime 1.29.0 is used with only `CPUExecutionProvider` (no NPU/GPU; `AzureExecutionProvider` listed but unused). The deduplicated-model sweep covers 12 configurations: 3 models × 2 quantizations (FP32/INT8 dynamic) × 2 thread counts (1 and 4 via `intra_op_num_threads`) × batch 1, with input 3×224×224. Each configuration runs 200 iterations after 10 warmup iterations (single session per config), reporting mean/std/P50/P95/P99 latency, throughput, RSS memory (mean/peak), CPU%, and die temperature before/after. Variability reported is within-session; variability across independent benchmark sessions was not measured. All timings are model-only (eval preprocessing and data transfer excluded), not end-to-end application throughput. Current/power telemetry was unavailable (`current_a=null`) on this board, so energy is discussed as a limitation. All 12/12 runs completed without thermal throttling.

IV. Results and Discussion

A. Classification Performance

Table I summarizes classification performance on the deduplicated test set. All three architectures exceed 97% accuracy, showing ImageNet transfer learning transfers well to the poultry fecal domain. Dynamic INT8 reduces accuracy slightly for MobileNetV2 and ShuffleNetV2; the paired test detects discordance for MobileNetV2 (b=18, c=7, p=0.0455) but not for ShuffleNetV2 (b=6, c=5, p=1.000). This test does not establish equivalence. The EfficientNet-B0 collapse is 88.13 percentage points.

TABLE I
Test accuracy on deduplicated split (1,728 images); dynamic INT8 PTQ (ORT). Macro-F1 (balanced acc.); brackets: 95% bootstrap percentile CI (B=10,000, seed 42).

Model	FP32 acc (%)	INT8 acc (%)	Drop (pp)	FP32 macro-F1 (bal. acc)	INT8 macro-F1 (bal. acc)
MobileNetV2	97.92	97.28	0.64	97.57 (97.26)	96.45 (96.09)
ShuffleNetV2	97.69	97.63	0.06	97.10 (97.34)	96.82 (96.76)
EfficientNet-B0	98.55 [97.97, 99.07]	10.42 [8.97, 11.92]	88.13	98.22 (98.03)	7.49 (26.89)

The EfficientNet-B0 collapse is characterized in Table II with selective ablations (Table III) and family SQNR diagnostics (Table IV).

TABLE II
EfficientNet-B0 static QDQ diagnostic (MinMax; first 100 class-ordered train images, Coccidiosis only). Balanced rows: 25/class replication (retrained model, FP32 98.15%).

Config	Test acc. (%)	vs FP32 b/c	vs FP32 p	vs dynamic b/c	vs dynamic p
Dynamic INT8	10.42	—	—	—	—
Static QDQ per-tensor	9.32	1548/7	$< 10^{-15}$	36/17	0.013
Static QDQ per-channel	94.56	75/7	1.37×10^{-13}	14/1468	$< 10^{-15}$
Static QDQ per-tensor (balanced)	27.14	—	—	—	—
Static QDQ per-channel (balanced)	93.40	—	—	—	—

TABLE III
Selective per-family static QDQ ablation for EfficientNet-B0 (MinMax, first 100 class-ordered training images, Coccidiosis only). Families are grouped from the ONNX graph; FP32 reference accuracy is 98.50%.

Config.		Size	Acc.	Collapse
A	All per-tensor	4.29	9.32	yes
B	All per-channel	4.67	94.56	no
C	**Depthwise** per-channel, rest per-tensor	4.33	**93.46**	no
D	All per-channel *except* depthwise	4.51	8.33	yes
E	Depthwise left in FP32	4.69	97.16	no
F	Squeeze-and-excitation left in FP32	5.91	12.44	yes

TABLE IV
Layerwise weight SQNR under simulated INT8. Values are family means; gain is the per-channel improvement. Per-layer medians are reported in Sec. IV-A.

Family	Tensors	Params	Per-tensor	Per-channel (gain)
Depthwise	16	182k	**27.97**	44.42 (+16.46)
SE reduce	16	314k	32.68	37.14 (+4.46)
SE expand	16	314k	34.00	45.26 (+11.26)
Pointwise	32	3.15M	33.67	42.30 (+8.63)

EfficientNet-B0 has the highest FP32 accuracy (98.55%) but collapses to 10.42% under dynamic INT8; Fig. 1 shows 1,676/1,728 predictions falling in the Newcastle Disease column, with per-class INT8 recall of 2.27% (Coccidiosis), 4.76% (Healthy), 0.54% (Salmonella), and 100% (Newcastle Disease), i.e. a single-class predictor. Static per-tensor QDQ also collapses (9.32%), whereas per-channel QDQ recovers 94.56% (Table II). A stratified 25/class replication on a retrained model (same 50-epoch protocol, seed 42; FP32 98.15%, 7 images below the main model) gives 27.14% per-tensor and 93.40% per-channel versus 9.32% (coincidentally identical to the main model) and 90.74% single-class on the same model: balanced calibration lifts per-tensor by 17.82 pp but the collapse persists, while per-channel stays within 2.66 pp of the single-class result, so the failure is not explained by calibration skew in this replication. A plausible mechanism is channel-wise range imbalance: a shared per-tensor scale is set by the largest channel, reducing resolution for smaller ones. A depthwise audit over 16, 17, and 19 layers gives median P95/P5 channel-range ratios of 39.94×, 9.20×, and 5.32× (`depthwise_range_audit.json`), and layer-level depthwise SQNR has medians of 16.67, 34.06, and 37.79 dB (worst layer 13.87, 30.17, and 35.29 dB), so EfficientNet-B0's depthwise family is uniformly worse rather than dominated by a few outliers. Its worst layer (480 channels, range ratio 112×) has channels at 0 dB, i.e. quantized to a constant, versus 12.23 dB for MobileNetV2's worst channel. The family mean (27.97 dB, Table IV) hides this spread; we hypothesize that severely quantized channels drive the collapse, which a single-layer ablation would test. Section IV-D discusses the accuracy-latency implications.

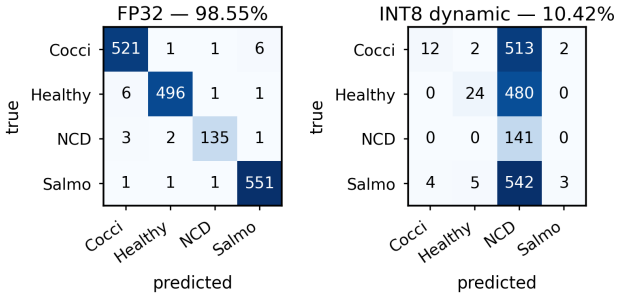


Fig. 1. EfficientNet-B0 confusion matrices on the deduplicated test set (1,728 images): FP32 (left) versus dynamic INT8 (right). Under dynamic INT8 the model degenerates to a near-single-class predictor. Matrices for MobileNetV2 and ShuffleNetV2 are provided in the artifact repository.

B. Quantization Impact

Table V presents the quantization analysis on the deduplicated split. All architectures achieve 3.3–3.7× model size reduction through INT8 quantization.

INT8 quantization increases latency by 3.0–16.8× versus FP32 on the x86 reference. Our reference CPU exposes AVX-VNNI yet still slows down, suggesting the ORT CPU kernel path (quantize/dequantize ops around the matmul) dominates; graph operators alone do not prove which kernels execute, so Sec. IV-C attributes this cautiously using executed-kernel profiling on RPi5.

TABLE V
INT8 QUANTIZATION ANALYSIS (WSL X86 REFERENCE, BATCH 1, 200 RUNS + 10 WARMUP, MODEL-ONLY TIMING, DEFAULT THREADS). DEDUPLICATED-SPLIT MODELS.

Model	Size		Latency		INT8/
	FP32	INT8	FP32	INT8	FP32
MobileNetV2	8.48	2.30	2.40	40.25	16.8×
ShuffleNetV2	4.89	**1.47**	4.15	12.62	3.0×
EfficientNet-B0	15.3	4.16	4.35	57.18	13.1×

ShuffleNetV2 degrades least (3.0× versus 16.8× for MobileNetV2). Channel-shuffle ops are non-parametric and less sensitive to precision, while MobileNetV2 depthwise separable convolutions are more sensitive.

C. Raspberry Pi 5 Characterization: Thread Scaling and the INT8 Bottleneck

Table VI and Table VII summarize the deduplicated-model Raspberry Pi 5 latency at batch=1. Fig. 2 visualizes the thread-scaling contrast. FP32 scales with threads: MobileNetV2 drops from 33.16 ms (1 thread, 30.2 FPS) to 14.23 ms (4 threads, 2.33× speedup); ShuffleNetV2 from 14.19 ms to 6.19 ms (2.29×); EfficientNet-B0 from 69.45 ms to 31.56 ms (2.20×).

The dynamic INT8 slowdown reported in Table VII is a property of dynamic quantization on this runtime, not of INT8 in general. The FP32 reference reads 98.50% here versus 98.55% in Table I because accuracy is evaluated on the exported ONNX graph; the 0.05-point gap is a single image. Table VI repeats the EfficientNet-B0 measurement in a separate session on the same board and build (≤0.7% from Table VII) with the static QDQ graphs of Table II. Static QDQ is 2.31–2.34× *faster* than FP32 at 4 threads (13.63 and 13.50 ms versus 31.53 ms), reversing the dynamic result (63.99 ms). The static per-channel result is faster and more accurate in this experiment, but it does not establish that granularity always dominates bit width: the comparison uses one board, one runtime, one seed, and one calibration set, and leaving depthwise layers in FP32 reaches 97.16%.

TABLE VI
EFFICIENTNET-B0 ON RASPBERRY PI 5, BATCH=1, MEAN OF 200 RUNS AFTER 10 WARMUP, CPUEXECUTIONPROVIDER ONLY, ORT 1.29.0. ACCURACY ON EXPORTED ONNX GRAPHS.

Quantization	Granularity	Size (MB)	t=1 (ms)	t=4 (ms)	Rel. FP32 t=4	Acc. (%)
FP32	—	15.30	69.00 (68.97)	31.53 (30.89)	1.00×	98.50
INT8 dynamic	per-tensor	4.16	76.38 (76.35)	63.99 (63.82)	0.49×	10.42
INT8 static	per-tensor	4.29	24.51 (24.50)	13.63 (13.37)	2.31×	9.32
INT8 static	**per-channel**	4.67	24.66 (24.64)	**13.50** (13.49)	2.34×	94.56

Dynamic INT8 is slower than FP32 at four threads (1.93–2.80×), and its thread scaling is limited: from 1 to 4 threads, INT8 improves only 1.10–1.19× versus 2.20–2.33× for FP32. This indicates limited parallelization of INT8 kernels in ONNX Runtime 1.29.0 on Cortex-A76. One exception is ShuffleNetV2 INT8 at one thread (13.65 ms), which is slightly faster than its FP32 counterpart (14.19 ms).

On the deduplicated RPi5 sweep (batch=1 only), runtime RSS is similar across quantizations for MobileNetV2 (∼99–108 MB) and ShuffleNetV2 (∼111–112 MB), so INT8 does not save RAM at inference on this stack—a practical caveat for memory claims.

TABLE VII
RASPBERRY PI 5 LATENCY, BATCH=1 (MEAN OF 200 RUNS, P50 IN PARENTHESES). DEDUPLICATED-SPLIT MODELS, ORT 1.29.0, 44–56°C, THROTTLED=0X0.

Model	Quant	t=1 (ms)	t=4 (ms)	Scaling 1→4	FP32→INT8 t=4
MobileNetV2	FP32	33.16 (33.00)	**14.23** (14.03)	2.33×	—
	INT8	43.88 (43.86)	39.83 (39.73)	1.10×	2.80× slower
ShuffleNetV2	FP32	14.19 (14.19)	**6.19** (6.18)	2.29×	—
	INT8	13.65 (13.64)	11.95 (11.76)	1.14×	1.93× slower
EfficientNet-B0	FP32	69.45 (69.42)	**31.56** (31.18)	2.20×	—
	INT8	76.68 (76.67)	64.39 (64.19)	1.19×	2.04× slower

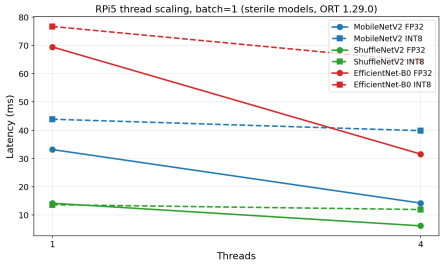


Fig. 2. Raspberry Pi 5 thread scaling at batch 1. FP32 benefits more from four threads, whereas dynamic INT8 shows limited scaling and remains slower at four threads.

TABLE VIII
NODE EXPANSION AND EXECUTED-KERNEL DISPATCH ON RPI5. DYNAMIC INT8 DISPATCHES CONVINTEGER AND QUANTIZATION-RELATED OPERATORS; FP32 HAS NO QUANTIZATION KERNELS. WITH THREE ARCHITECTURES, THE MEASUREMENTS SUPPORT THE OBSERVED SLOWDOWN RANKING BUT NOT A GENERAL EXPANSION CORRELATION.

Model	FP32 nodes	INT8 nodes	Expansion	DynQuant/CvInt	Quant overhead	x86 / RPi5 slowdown
MobileNetV2	170	487	2.86×	53 / 52	29.2%	16.8× / 2.80×
ShuffleNetV2	902	1240	1.37×	54 / 56	20.7%	3.0× / 1.93×
EfficientNet-B0	239	730	3.05×	82 / 81	24.9%	13.1× / 2.04×

Measured bottleneck. Table VIII quantifies why INT8 is slower. Dynamic PTQ does not insert static QuantizeLinear/DequantizeLinear nodes; it expands each Conv (52–81) into a DynamicQuantizeLinear + ConvInteger + Mul (scale) + Cast pattern, so node count grows by 1.37–3.05× on the deduplicated graphs. ORT kernel profiling on RPi5 (enable_profiling, 15+5 runs) shows dispatched kernels: FP32 has zero quantization kernels, while INT8 dynamic dispatches ConvInteger 51–58%, DynamicQuantizeLinear 9–15%, and Mul/Add/Cast 20–35% combined; all quantization-related kernels add 20.7–29.2% overhead. The slowdown ranking agrees with the measured overhead ranking for these three models, but the sample is too small to claim a general expansion-overhead correlation. CPU feature detection exposes ARM dot-product support (`asimddp`) but not `i8mm` or SVE; whether the MLAS kernels use the dot-product instructions was not verified. Artifact `kernel_dispatch_artifact.json` contains the per-operator timings and operation counts, complementing

TinyML system studies [16].

D. Accuracy-Latency Tradeoff

ShuffleNetV2 FP32 is fastest on RPi5 at 6.19 ms, EfficientNet-B0 is most accurate at 98.55%, and MobileNetV2 FP32 is intermediate (97.92%, 14.23 ms). The best and worst FP32 accuracies differ by only 0.86pp, so hardware constraints and quantization scheme should drive model choice. On x86, ShuffleNetV2 FP32 (4.15 ms) is slower than MobileNetV2 (2.40 ms) but faster on RPi5; its 902 graph nodes versus 170 (Table VIII) make per-node overhead a plausible, untested cause on the faster x86 core, so deployment claims rest on RPi5 only.

E. Comparison with Related Work

Our accuracy (98.55%) is competitive with prior results on the same dataset family, although the protocols differ in splits, deduplication, and metrics, so this is not a head-to-head benchmark.

We do not claim depthwise sensitivity as a new finding: Sheng *et al.* already showed separable convolutions quantize poorly [17], per-channel weight quantization is the documented remedy [18], and cross-layer equalization [19] targets the same range imbalance we measure. Our contribution is narrower and empirical: a case study showing the failure appears in EfficientNet-B0 but not in MobileNetV2 or ShuffleNetV2 under an identical PTQ pipeline, plus the runtime behaviour of dynamic INT8 on ONNX Runtime for Cortex-A76 (1.93–2.80× slower than FP32 at four threads).

V. Conclusion

This paper characterizes lightweight CNN architectures with INT8 quantization for poultry fecal disease detection. Dynamic INT8 preserves MobileNetV2 and ShuffleNetV2 but collapses EfficientNet-B0 to 10.42%; static per-channel QDQ recovers it to 94.56%, and the ablation indicates strong sensitivity to depthwise-layer granularity (93.46% with depthwise per-channel versus 8.33% without). Dynamic INT8 is 3.0–16.8× slower on x86 and 1.93–2.80× slower on RPi5, whereas static per-channel QDQ is 2.34× faster than FP32 on RPi5.

This study covers one board, one runtime build, one dataset, one seed, and a single-class (Coccidiosis-only) 100-image calibration set; multi-seed intervals, calibration-size sweeps, and paired McNemar tests for the stratified balanced replication are absent from the reported tables. Energy was not measured and RPi5 latency is model-only at batch 1. The selective ablation covers EfficientNet-B0 only; extending it to MobileNetV2 and ShuffleNetV2 would test whether the depthwise sensitivity generalizes. Quantization-aware training and class-imbalance mitigation for the underrepresented NCD class remain open.

Code and deduplication artifacts (audit, split manifests, evaluation JSONs, benchmark logs, plotting scripts) are archived at Zenodo: https://doi.org/10.5281/zenodo.23374216. The corresponding source repository is https://github.com/jaweed3/paper_poultry_p2.

Acknowledgment

This article is an output of the Scientific Journal Writing Training Program organized by Rumah Jurnal UNIDA Gontor and funded by Universitas Darussalam Gontor through the Institute for Research and Innovation (LPI).

References

[1] A. Dhungana, X. Yang, B. Paneru, S. Dahal, G. Lu, and L. Chai, "An integrated deep learning approach for poultry disease detection and classification based on analysis of chicken manure images," *AgriEngineering*, vol. 7, no. 9, p. 278, 2025. DOI: 10.3390/agriengineering7090278.

[2] D. Machuve *et al.*, "Poultry diseases diagnostics models using deep learning," *Frontiers Artif. Intell.*, vol. 5, p. 733345, 2022. DOI: 10.3389/frai.2022.733345.

[3] A. Tasdelen and Y. Arslan, "Detection of high-risk diseases in poultry feces through transfer learning," *Eng. Sci. Technol. Int. J.*, vol. 64, p. 102002, 2025. DOI: 10.1016/j.jestch.2025.102002.

[4] M. Z. Degu and G. L. Simegn, "Smartphone based detection and classification of poultry diseases from chicken fecal images using deep learning techniques," *Smart Agric. Technol.*, vol. 4, p. 100221, 2023. DOI: 10.1016/j.atech.2023.100221.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.

[6] N. Ma, X. Zhang, H.-T. Huang, and J. Sun, "ShuffleNetV2: Practical guidelines for efficient CNN architecture design," in *Proc. ECCV*, 2018, pp. 116–131.

[7] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019, pp. 6105–6114.

[8] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.

[9] T.-Y. Chi, "FecalFed: Privacy-preserving poultry disease detection via federated learning," *arXiv:2604.00559*, 2026.

[10] U.S. Department of Agriculture, National Agricultural Statistics Service, "Poultry production and value: 2023 summary," Apr. 2024. [Online]. Available: https://www.nass.usda.gov/Publications/Todays_Reports/reports/plyaan24.pdf

[11] T.-M. Hoang *et al.*, "A comprehensive evaluation of lightweight deep learning models for tomato disease classification on edge computing environments," *Sci. Rep.*, vol. 16, p. 12320, 2026. DOI: 10.1038/s41598-026-42439-6.

[12] Microsoft, "Quantize ONNX models," ONNX Runtime Documentation, 2024. [Online]. Available: https://onnxruntime.ai/docs/performance/quantization.html

[13] D. Machuve *et al.*, "Machine learning dataset for poultry diseases diagnostics," Zenodo, 2022. DOI: 10.5281/zenodo.4628934.

[14] M. Nagel *et al.*, "A white paper on neural network quantization," *arXiv:2106.08295*, 2021.

[15] J. Deng *et al.*, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE CVPR*, 2009, pp. 248–255.

[16] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML Systems," in *Proc. Machine Learning and Systems (MLSys)*, 2021, pp. 800–811.

[17] T. Sheng, C. Feng, S. Zhuo, X. Zhang, L. Shen, and M. Aleksic, "A quantization-friendly separable convolution for MobileNets," in *Proc. 1st Workshop Energy Efficient Mach. Learn. Cogn. Comput. Embedded Appl. (EMC2)*, 2018. arXiv:1803.08607.

[18] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv:1806.08342*, 2018.

[19] M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling, "Data-free quantization through weight equalization and bias correction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1325–1334, doi: 10.1109/ICCV.2019.00141.